Operating Systems

Grado en Informática 2026/2027

Lab Assignment 2: Memory

We continue to code the shell we started in the first lab assignment. Now the shell will have the ability to allocate or deallocate memory blocks via malloc, mapping files or shared memory. The shell will keep a list ot the memory blocks allocated (only the ones allocated with the command *allocat*e. not the ones it needs to allocate for its normal working).

Command	Param	
allocate	[-malloc|-shared| -createshared|-mmap] ...	allocates a block of memory (malloc, shared memory, or mmapped file). Without parameters shows the list of allocated blocks
	-malloc siz	allocates a block of memory of size siz using malloc. It adds information on the block to the shell's list
	-createshared cl siz	creates a new block of shared memory of key cl and size siz and attaches it to the shell's address space. cl must not be in use. It adds information on the block to the shell's list
	-shared cl	attaches an existent block of shared memory with key cl to the shell's address space. It adds information on the block to the shell's list
	-mmap fil perm	maps the file fil to memory (starting at offset 0 and for the length of the file) with permissions perm. It adds information on the block to the shell's list
deallocate	[-malloc|-shared|-delkey| -mmap|addr] ...	deallocates a block of memory (malloc, shared memory, or mmapped file).It eliminates the block from the shell's list. Without parameters shows the list of allocated blocks
	-malloc siz	deallocates a previously allocated (with malloc) block of memory of size siz using malloc. It deletes information on the block from the shell's list
	-shared cl	detaches apreviously attached block of shared memory with key cl from the shell's address space. It deletes information on the block from the shell's list
	-mmap fil	unmaps the file fil from memory and deletes information on the corresponding block from the shell's list
	-delkey cl	deletes key cl from the system. Does not detach anything
	addr	deallocates block at address addr (provide it was previouls allocated with allocate) and deletes the corresponding information from the shells list
i-o	[read|write] [-o] f addr cnt	direct input output from memory to files
	read f addr cnt	reads cnt bytes from file f into memory address addr (in hexadecimal). Does not check the value of addr so it could produce runtime errors
	write [-o] f addr cnt	writes cnt bytes from memory address addr (in hexadecimal) into file f. if -o is present the file can be overwritten
memory	[-blocks|-funcs|-vars| -all|-pmap]	shows information on the shell's memory
	-blocks	prints the list of allocated (malloc, shared or mmap) memory blocks in the shell
	-funcs	prints the addresses of three of the shell's functions and three library functions called within the shell
	-vars	prints the addresses of three of the shell's automatic variables (both initialized and not initialized), three static variables (both initialized and non initialized) and three external variables.
	-all	equivalent to -blocks -funcs -vars
	-pmap	shows the shell's address space (using the command pmap or equivalent)

read	df addr cont	transfers cont bytes from the openfile df to the address addr. addr is expressed in HEX and need not be a valid address.
write	df addr cont	transfers cont bytes from addr to the open file described by df. addr is expressed in HEX and need not be a valid address.
recurse	**n**	executes the recursive function n times. This funcion has an automatic array of size 2048, a static array of size 2048 and prints the addresses of both arrays and its parameter (as well as the number of recursion) before calling itself
memfill	addr cont hcode	fills cont bytes starting at memory address addr with hex value hcode. Does not check addr nor addr+cont so it could produce runtime errors
memdump	addr cont	dumps onto screen cont bytes starting at memry address addr. Dumps both the hex codes and the corresponding character. Does not check addr nor addr+cont so it could produce runtime errors

IMPORTANT:

We have to implement (list implementation free) a list of memory blocks. For each block we must store

- Its memory address
- Its size
- Time of allocation
- Type of allocation (malloc memory, shared memory, mapped file)
- Other info: key for shared memory blocks, name of file and file descriptor for mapped files.

The shell commands ***allocate,*** *and* ***deallocate*** allocate and deallocate memory blocks and add (or remove) them from the list. Each element on the list has info of a memory block we created with the shell command ***allocate*** The information about that block is the one previously stated. We'll deal with three types of memory blocks.

- **malloc memory.** this is the most common memory we use, we allocate it with the library function *malloc* and deallocate it with the library function *free*

- **shared memory.** this is memory that can be shared among several processes. The memory is identified by a number (called key) so that two (or more) processes using the same key get to the same block of memory. We use the system call *shmget* to obtain the memory and *shmat* and *shmdt* to place it in (or detach it from) the process address space. shmget needs the key, the size and the flags. We'll use *flags=IPC_CREAT | IPC_EXCL| permisions* to create a new one (gives error if it already exists) and *flags=permisions* to use an already created one. To delete a key we'll use the **shared -delkey** command (this command deallocates nothing, just deletes the key). Status of the shared memory in the system can be checked via the command line with the *ipcs* command. An additional C file (ayudaP2-27.txt) is provided with some useful functions

- **mapped files.** We can also map files in memory so that they appear in the address space of a process. System calls mmap and munmap do the trick. Again, the additional C file (ayudaP2-27.txt) is provided with some useful functions

The contents of our list must be compatible with what the system shows with the *pmap* command (*procstat vm*, *vmmap* ...depending on the platform)

The recursive function has a static and a dynamic array for the same size (2048 bytes) and prints their addresses together with the parameter address and value

LEGITIMATE RUNTIME ERRORS

Although NO RUNTIME ERROR WILL BE ALLOWED (segmentation, bus error . . .) and programs with runtime errors will yield no score, this program can legitimately produce segmentation fault errors in scenarios such as these:

- **memdump** or **memfill** try to access an invalid address supplied through the command line
- **memfill, i-o, read** or **write** corrupt the user stack or the heap

REMEMBER:

- Information on the system calls and library functions needed to code these programs is available through man: (shmget, shmat, malloc, free, mmap, munmap, shmctl, open, read, write, close, sscanf. . .)
- A reference shell is provided (for various platforms) for students to check how the shell should perform. This program should be checked to find out the syntax and behaviour of the various commands. **PLEASE DOWNLOAD THE LATEST VERSION** (the *version* command, on newer reference shell, shows which version you are using)
- The program should compile cleanly (produce no warnings even when compiling with gcc -Wall)
- These programs can have no memory leaks (you can use valgrind to check)
- When the program cannot perform its task (for whatever reason, for example, lack of privileges) it should inform the user (See errors section)
- All input and output is done through the standard input and output
- Errors should be treated as in the previous lab assignment

WORK SUBMISSION

- Work must be done in pairs.
- Moodle will be used to submit the source code: a zipfile containing a directory named P2 where all the source files of the lab assignment reside
- The name of the main program will be p2.c, Program must be able to be compiled with gcc p2.c, Optionally a Makefile can be supplied so that all of the source code can be compiled with just make. Should that be the case, the compiled program should be called p2 and placed on the same directory as the sources (no build directories or similar)
- ONLY ONE OF THE MEMBERS OF THE GROUP will submit the source code. The names and logins of all the members of the group should appear in the source code of the main programs (at the top of the file)
- **Works submited not conforming to these rules will be disregarded**.
- DEADLINE: 23:00, Friday November the 6th, 2026

INTERESTING THINGS TO DO WITH THIS LAB ASSIGMENT

* Allocate 3Gb of memory with malloc. Check where it is on the address space

* Allocate shared memory, read the shell's code file into that address. On other terminal execute another copy of the shell, allocate shared memory with the same key. Dump the contents of that memory to screen

*Check how many iterations of the recursive function are necessary to overflow the stack. Change the size of the automatic and/or static array to 4096 (instead of the 2048). How many iterations are now neccesary to overflow the stack

*Map a text file to memory, check address space of the process, dump the address where the file is mapped. If you get "segmentation fault".... have you checked the mapping permissions?

*Try to dump or fill (with memdump or memfill) addresses that are not in the shell's address space

*Compile the shell with the *-static* flag, check the difference of address spaces. Check where on those addresses spaces the different variables and functions are

*(In linux) Allocate with malloc a block bigger than 256k, (for example 500000000), linux will return an address such as 0xNNNNNNNNNNNNN010, and the pmap command would show NNNNNNNNNNNNN000. Fill the memory at 0xNNNNNNNNNNNNN000 with any character (for at least 16 bytes). Now free the block (malloc -free) at 0xNNNNNNNNNNNNN010